Lab 10

More about Functions in C++

Objectives

The objective of this lab is to:

We will learn and apply the

- Function Call by Value
- Function Call by Reference
- Default Values in Parameters
- Syntax
- Arguments

Calling Function: Call By Value

Call by value is used when whenever the called function does not need to modify the value of the caller's original variable. In this case a copy of the variable is passed to the function and when the function changes its values then it has no effect on the value of the variable in the main function. Note that & (ampersand)is called "addressof"operator. It is used to mention the address of a variable. For example int &num describes the address of the variable num.

Example:

```
#include
using namespace std;
void func(int);                    // function prototype
int main() {
 int i = 10;
 func(i);                          //Function call
 cout << "The value of i remains " << i << endl;
 return 0;  }
// function definition
void func(int i) {
 15 i = i + 10;
 cout << "Inside function , the value of i changed to " << i << endl;}
```

Output:

```
Inside function, the value of i changed to 20
The value of i remains 10
```

Example:

```
#include

using namespace std;

void swapping(int, int); // function prototype

int main() {

  int x = 50, y = 70;

  cout << "Before calling function: values are: ";

  cout << "x = " << x << " and " << "y = " << y << endl;

  swapping(x, y);     //Function call

  cout << "After calling function: values are: ";

  cout << "x = " << x << " and " << "y = " << y << endl;

  return 0; }

// function definition

 void swapping(int x1, int y1) {

  int z1;
```

```
z1 = x1;

x1 = y1;

y1 = z1;

cout << "Values inside function: ";

cout << "x1 = " << x1 << " and " << "y1 = " << y1 << endl; 25 }
```

Output:

```
Before calling function: values are: x = 50 and y = 70

Values inside function: x1 = 70 and y1 = 50

After calling function: values are: x = 50 and y = 70
```

2 Calling Function:

Call by Reference If you are calling by reference it means that compiler will not create a local copy of the variable which you are referencing to. It will get access to the memory where the variable saved. When you are doing any operations with a referenced variable you can change the value of the variable.

Example:

```
#include
using namespace std;
void swapping(int, int);        // function prototype
int main() {
 int x = 50, y = 70;
 cout << "Before calling function: values are: ";
 cout << "x = " << x << " and " << "y = " << y << endl;
```

```
swapping(x, y);          //Function call
 cout << "After calling function: values are: ";
 cout << "x = " << x << " and " << "y = " << y << endl;
 return 0: }



// function definition
void swapping(int x1, int y1)
{
  int z1;   z1 = x1;   x1 = y1;   y1 = z1;
  cout << "Values inside function: ";
  cout << "x1 = " << x1 << " and " << "y1 = " << y1 << endl;}
```

Output:

```
Before calling function: values are: x = 50 and y = 70
Values inside function: x1 = 70 and y1 = 50
After calling function: values are: x = 50 and y = 70
```

2 Calling Function:

Call By Reference If you are calling by reference it means that compiler will not create a local copy of the variable which you are referencing to. It will get access to the memory where the variable saved. When you are doing any operations with a referenced variable you can change the value of the variable.

Example:

```
#include

using namespace std;

void swapping(int &, int &);      // function prototype

int main() {

 int x = 50, y = 70;

 cout << "Before calling function: values are: ";

 cout << "x = " << x << " and " << "y = " << y << endl;

 swapping(x, y);                          //Function call by reference

 cout << "After calling function: values are: ";

 cout << "x = " << x << " and " << "y = " << y << endl;

 return 0; }

// function definition

void swapping(int &x1, int &y1) {

  int z1; z1 = x1;

  x1 = y1;

  y1 = z1; }
```

.

Output:

```
Before calling function: values are: x = 50 and y = 70

After calling function: values are: x = 50 and y = 70
```

Passing by reference is also an effective way to allow a function to return more than one value. For example, here is a function that returns the previous and next numbers of the first parameter passed.

3 Default Values in Parameters

When declaring a function we can specify a default value for each of the last parameters. This value will be used if the corresponding argument is left blank when calling to the function. To do that, we simply have to use the assignment operator and a value for the arguments in the function declaration. If a value for that parameter is not passed when the function is called, the default value is used, but if a value is specified this default value is ignored and the passed value is used instead.

Example:

```
#include
using namespace std;
// function definition
 int divide(int a = 8, int b = 2) {
   int r; 8 r = a / b;    return (r); }
int main() {
  cout << divide();       //Function call
  cout << endl;
  cout << divide(12);
  cout << endl;
  cout << divide(63, 7);   //Function call by value
return 0; }
```

Output:

```
6
4
9
```

Lab10: Lab Tasks

1:Function with Parameters

- Declare a function called ”multiply” that takes two integer parameters:”num1”and”num2”. Inside the function, calculate the product of the two numbers.
- Display the result within the function.
- Call the function from the main program, passing two numbers, to perform the multiplication.

```
#include <iostream>            // Header file for input and output
using namespace std;           // Allows use of standard names
                               like cout and endl
void multiply(int num1, int num2)
// Function definition for multiplication
{
int product = num1 * num2;     // Calculate the product of two numbers
cout << "Product = " << product << endl;       // Display the product
}
int getSquare(int number)      // Function definition to calculate square
{
```

```
int square = number * number;   // Calculate square of the number
return square;                  // Return the square value
}
int main()                      //  Main function where program execution
                                    starts
{
multiply(5, 4);             // Call multiply function and pass two numbers
 int result;                // Declare variable to store returned value
result = getSquare(6); // Call getSquare function and store returned value
cout << "Square = " << result << endl;    // Display the square result
return 0;                                 // Indicate successful program
                                              termination
}
```

2. Function with Return Value

- Declare a function called ”getSquare” that takes an integer parameter: ”number”.
- Inside the function, calculate the square of the number.
- Return the result.
- Call the function from the main program, passing a number, and display the square. " write each line with comments

```
#include <iostream>        // Header file for input and output
using namespace std;
// Function definition for multiplication
void multiply(int num1, int num2)
{
   // Calculate the product of two numbers
    int product = num1 * num2;
   // Display the product

   cout << "Product = " << product << endl;
}
// Function definition to calculate square
int getSquare(int number)
{
// Calculate square of the number
  int square = number * number;
// Return the square value
  return square;
}
// Main function where program execution starts
int main()
{
 // Call multiply function and pass two numbers
```

```
multiply(5, 4);
 // Declare variable to store returned value
    int result;
 // Call getSquare function and store returned value
     result = getSquare(6);
 // Display the square result
     cout << "Square = " << result << endl;
// Indicate successful program termination
     return 0;
}
```